Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №4

По дисциплине: «Интеллектуальный анализ данных»

Тема: «Предобучение нейронных сетей с использованием RBM»

Выполнил:

Студент 4 курса

Группы ИИ-26

Ковальчук А. И.

Проверил:

Никитин С. К.

Брест 2026

Цель работы

Научиться осуществлять предобучение нейронных сетей с помощью ограниченных машин Больцмана (RBM – Restricted Boltzmann Machine).

Общее задание

1. Взять за основу нейронную сеть из лабораторной работы № 3. Выполнить обучение с предобучением, используя стек ограниченных машин Больцмана, алгоритм которого изложен в лекции. Условие останова при обучении отдельных слоёв как RBM выбрать самостоятельно.

2. Сравнить результаты, полученные при обучении без предобучения (ЛР 3); обучении с предобучением автоэнкодерным подходом (ЛР 3); обучении с предобучением, используя RBM.

3. Обучить модели на данных из ЛР 2, сравнить результаты по схеме из пункта 2.

4. Сделать выводы, оформить отчёт, загрузить исходный код и отчёт в репозиторий.

Вариант 7. Датасет HCV for Egyptian patients (UCI, 1385 объектов, 28 признаков). Задача — классификация Baselinehistological staging (F1–F4). По п. 3 использован датасет грибов из ЛР 2 (8124 объекта, 117 признаков после one-hot, классы

edible / poisonous). Реализация — PyTorch, исходники RBM — из лекции. Seed = 42, дополнительно 43 и 44.

Ход работы

Разбиение 70/15/15 со стратификацией. StandardScaler обучался только на train. Архитектура та же, что в ЛР 3: вход → 128 → 64 → 32 → 16 → число классов, ReLU, Dropout, голова на логиты, CrossEntropy, Adam. Три режима на одном сплите: baseline — случайные веса; ae — жадные автоэнкодеры (25/15 эпох); rbm — стек RBM с CD-1. Для RBM условие останова — 40 эпох на HCV и 25 на грибах, момент 0,5 → 0,9 после пятой эпохи, weight_loss = $2 \cdot 10^{-4}$. Первый слой — гауссовский видимый (Identity), дальше Logistic. Голова не предобучается. Затем обучение головы при заморозке и fine-tune всей сети шагом $3 \cdot 10^{-4}$. Early stopping по val macro-F1.

Главная логика (RBM по лекции)

Класс RBM и шаг CD-1 перенесены из исходников лекции на тензоры PyTorch. Веса инициализируются как 0,1·N(0,1), скрытые пороги равны −1, видимые — нули. Для Logistic скрытый вектор сэмплируется сравнением с равномерным шумом.

```
class RBM:
    def __init__(self, shape, act_func):
        n_vis, n_hid = shape
        self.weights = 0.1 * torch.randn(n_vis, n_hid)
        self.vis_biases = torch.zeros(1, n_vis)
        self.hid_biases = -1.0 * torch.ones(1, n_hid)
        self.act_func = act_func

    def hid_activate(self, data):
        return self.act_func[1].apply(data @ self.weights + self.hid_biases)
```

```
    def vis_activate(self, data):
        return self.act_func[0].apply(data @ self.weights.T + self.vis_biases)

# один шаг CD-1, как в лекции (RBM_pretrain.train)
v1 = batch
h1 = rbm.hid_activate(v1)
h_sampled = (h1 > torch.rand_like(h1)).float()   # Logistic
v2 = rbm.vis_activate(h_sampled)
h2 = rbm.hid_activate(v2)
wu_vh = wu_vh * momentum + rate * ((v1.T @ h1 - v2.T @ h2) / bs)
bu_v  = bu_v  * momentum + rate / bs * (v1.sum(0) - v2.sum(0))
bu_h  = bu_h  * momentum + rate / bs * (h1.sum(0) - h2.sum(0))
rbm.weights += wu_vh - rate * weight_loss * rbm.weights
rbm.vis_biases += bu_v
rbm.hid_biases += bu_h
```

Жадный стек — функция pretrain_deep_network из лекции: каждый следующий RBM видит скрытые активации предыдущего. Веса $W^\top$ и скрытые пороги копируются в nn.Linear MLP.

```
architecture = [128, 64, 32, 16, n_classes]
# RBM1: Identity⇄Logistic (гауссовый видимый слой)
# RBM2..4: Logistic⇄Logistic
stack, hist = pretrain_deep_network(X_train, params, architecture, activ)
# голова last_rbm создаётся, но CD по ней не идёт — как в лекции
for linear, rbm in zip(model.encoders, stack):
    linear.weight.copy_(rbm.weights.T)
    linear.bias.copy_(rbm.hid_biases.squeeze(0))
# голова → fine-tune всей сети меньшим шагом
```

Результаты

HCV. Классы почти равны (336 / 332 / 355 / 362). Пропусков нет. Максимальная |корреляция| признака со стадией равна 0,057. Случайный уровень accuracy на 4 классах ≈ 0,25.

Таблица 1 – HCV, тестовая выборка, seed = 42

Метод	Accuracy	Macro-F1	train F1	val F1
baseline	0,231	0,232	0,934	0,231
AE (ЛР 3)	0,288	0,288	0,447	0,228
RBM	0,197	0,185	0,278	0,251

Таблица 2 – HCV, среднее ± ст. откл. по сидам 42, 43, 44

Метод	Accuracy	Macro-F1
baseline	0,264 ± 0,032	0,260 ± 0,028
АЕ (ЛР 3)	0,237 ± 0,050	0,230 ± 0,059
RBM	0,244 ± 0,048	0,196 ± 0,043

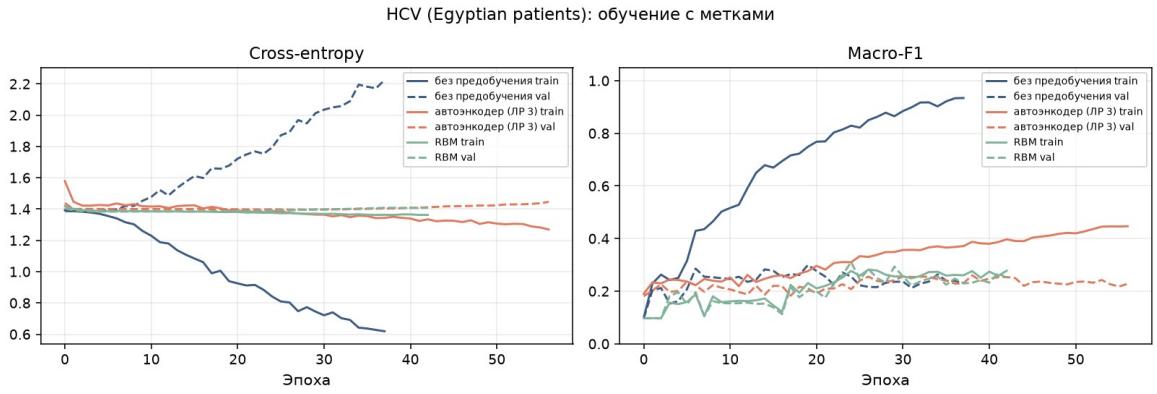


Рисунок 1 – HCV: кросс-энтропия и macro-F1 на train/val

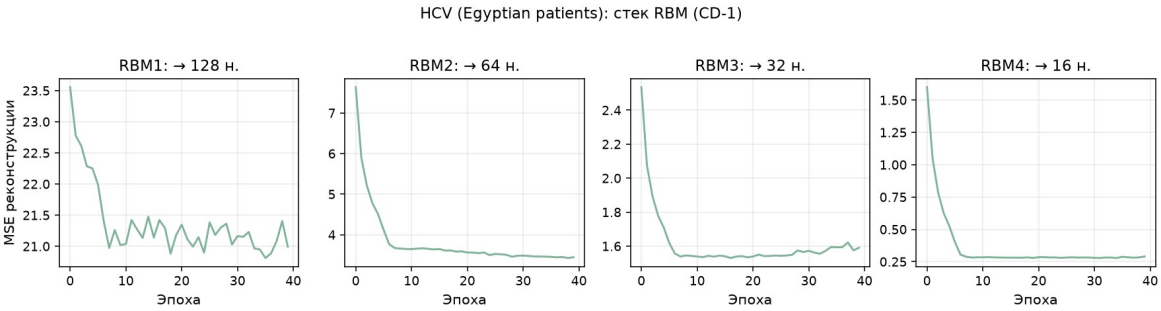


Рисунок 2 – HCV: ошибка реконструкции стека RBM

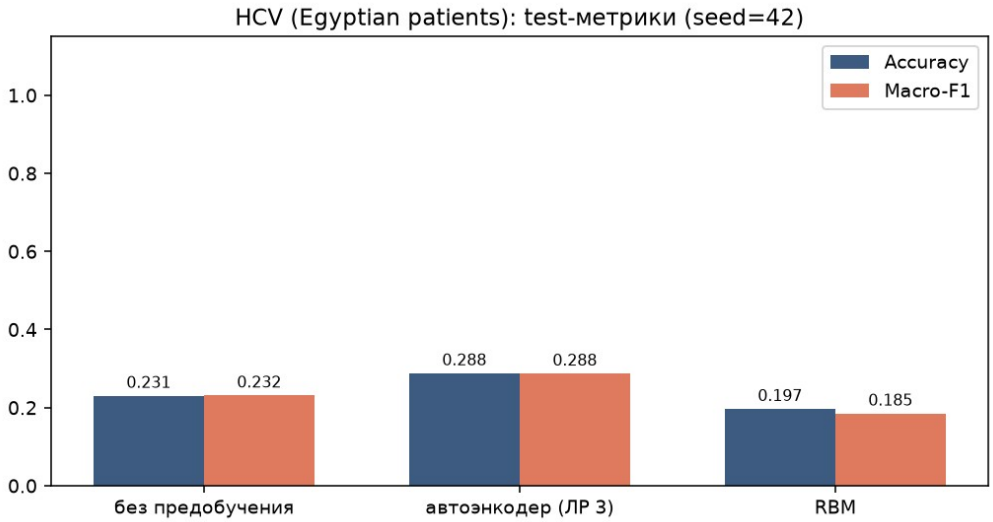


Рисунок 3 – HCV: сравнение трёх методов на тесте, seed = 42

Грибы (ЛР 2). Задача линейно разделима после one-hot. Все три модели на полном наборе дают F1 ≈ 1. Предобучение итоговую метрику почти не меняет — сравнивать методы по качеству нечего, качество высокое у всех.

Таблица 3 – Грибы, тестовая выборка, среднее по трём сидам

Метод	**Accuracy**	**Macro-F1**
baseline	0,9995 ± 0,0005	0,9995 ± 0,0005
AE (ЛР 3)	0,9995 ± 0,0005	0,9995 ± 0,0005
RBM	0,9992 ± 0,0008	0,9992 ± 0,0008

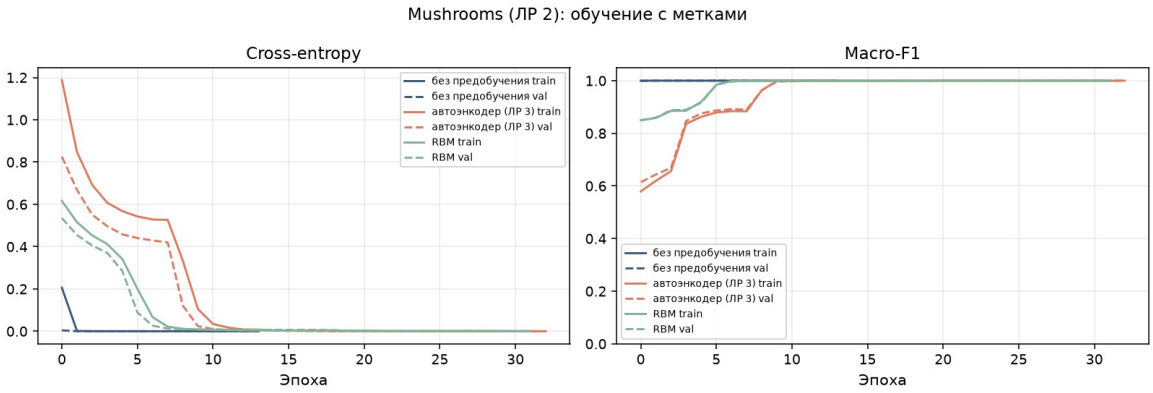


Рисунок 4 – Грибы: обучение трёх методов

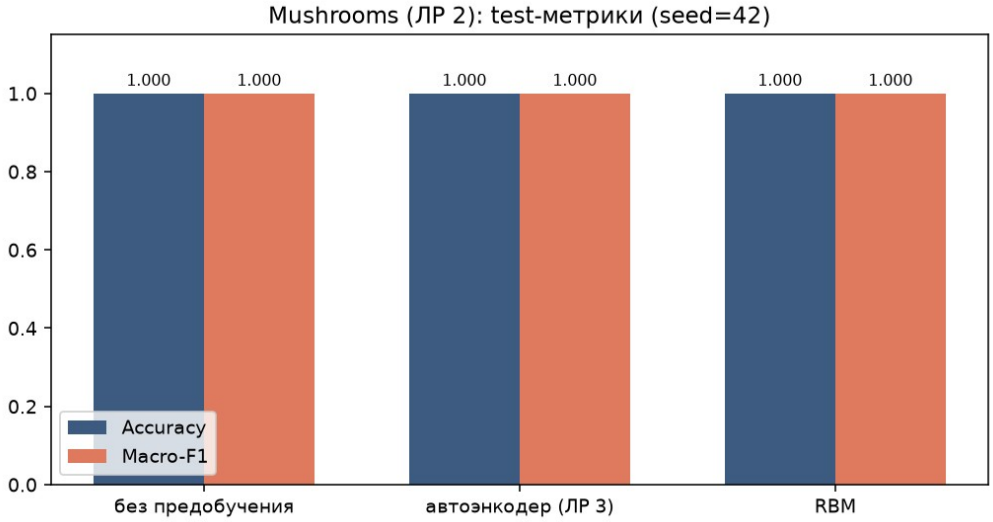


Рисунок 5 – Грибы: test-метрики, seed = 42



Рисунок 6 – Зазор train F1 − val F1

Выводы

1. На HCV все три модели дают accuracy в районе 0,19–0,30 (случайный уровень ≈ 0,25). Признаки слабо связаны со стадией, ни автоэнкодер, ни RBM F1 существенно не поднимают.

2. Без предобучения сеть переобучается: высокий train F1 при val F1 ≈ 0,23. Автоэнкодерный и RBM-старт держат меньший зазор train/val (рисунок 6).

3. На грибах из ЛР 2 все три модели дают macro-F1 ≈ 1,00. Это высокий результат, но он определяется самой задачей, а не выбором предобучения.

4. RBM по алгоритму лекции — рабочая альтернатива автоэнкодерам: та же жадная схема, другая целевая функция (CD вместо MSE). На данных варианта 7 оба способа проявляются как более устойчивое обучение, а не как рост тестового F1.